

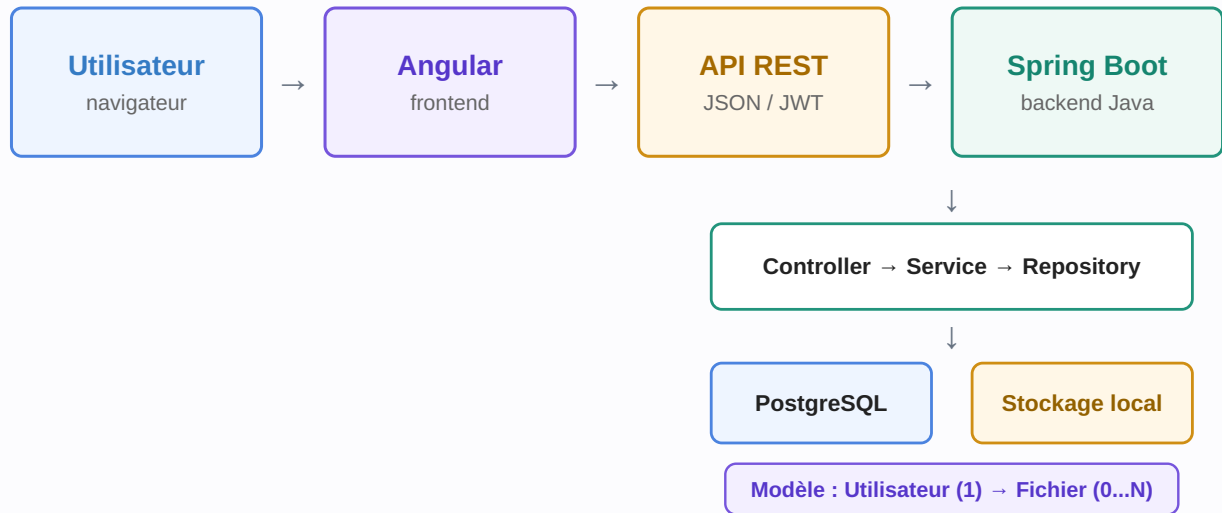
Documentation technique - DataShare

Projet 3 - Pilotez le développement d'une solution informatique | MVP de transfert sécurisé de fichiers

1. Architecture de l'application

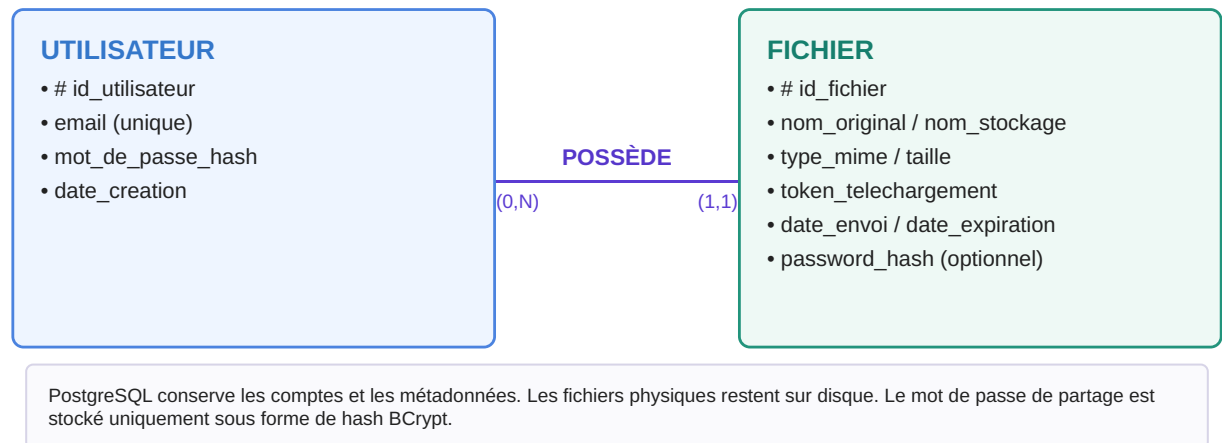
Architecture technique du MVP DataShare

Lire le schéma en suivant une requête d'upload



- Angular gère l'interface et les appels HTTP ; Spring Boot applique les règles métier et orchestre l'accès aux données.
- PostgreSQL conserve les comptes et métadonnées ; le système de fichiers local conserve les fichiers physiques.

2. Modèle de données



- Un utilisateur peut posséder 0 à plusieurs fichiers ; chaque fichier du MVP appartient à un utilisateur.
- Le modèle reste limité au MVP réellement livré et intègre la protection optionnelle des fichiers par mot de passe.
- Flyway V3 : index owner_id et expires_at pour l'historique et la purge.

Justification des choix technologiques

1 / 2 - Front-end et back-end

Principe de décision

Pour ce MVP, j'ai privilégié des technologies que je maîtrise déjà lorsqu'elles répondaient correctement au besoin. Cela m'a permis de consacrer le temps du projet aux fonctionnalités, à la qualité, aux tests et à la sécurité.

Front-end - Angular

Pourquoi je l'ai choisi : « J'ai choisi Angular car je l'ai déjà utilisé dans le projet 2. Je connais déjà les composants et les services, donc je peux avancer plus facilement. »

- Continuité avec le projet 2 : composants, services et structure générale déjà maîtrisés.
- TypeScript apporte du typage et rend les échanges avec l'API plus lisibles et plus faciles à maintenir.
- Angular structure les pages du MVP : inscription, connexion, upload, historique et téléchargement.
- L'interceptor ajoute le JWT, gère HTTP 401 (session effacée et redirection) ; le guard protège les routes privées.

Limite assumée

Angular est plus structurant qu'un frontend minimaliste. Ce coût est acceptable ici car l'outil était déjà connu et n'a pas ralenti la livraison du MVP.

Back-end - Spring Boot / Java 21

Pourquoi je l'ai choisi : « J'ai choisi Spring Boot avec Java car je l'ai déjà utilisé dans le projet 2. Je connais l'organisation Controller / Service / Repository et je suis à l'aise avec cet outil. »

- Controller gère HTTP, Service porte les règles métier, Repository accède aux données.
- Spring Security gère l'authentification JWT ; le backend reste stateless.
- Spring Data JPA simplifie la persistance PostgreSQL et Flyway versionne le schéma.
- Java 21 fournit un environnement stable ; BCrypt protège les mots de passe utilisateurs et de partage.

Bénéfice pour DataShare

Cette séparation facilite le diagnostic, les tests et la maintenance, et permet d'expliquer clairement le rôle de chaque couche.

Répartition des responsabilités

Couche	Responsabilité	Exemple DataShare
Angular	Interface et appels HTTP	Upload, historique, téléchargement
Controller	Entrées HTTP et statuts	/api/files/upload, /api/download/{token}/file
Service	Règles métier	Validation, expiration, mot de passe
Repository	Persistance JPA	UserRepository, StoredFileRepository

Justification des choix technologiques

2 / 2 - Base de données, stockage et outils complémentaires

Base de données - PostgreSQL

Pourquoi je l'ai choisi : « J'ai comparé PostgreSQL et MongoDB. PostgreSQL est plus naturel ici car les données sont structurées et liées : un utilisateur peut avoir plusieurs fichiers. »

- Le besoin métier contient une relation claire entre UTILISATEUR et FICHIER, adaptée à un modèle relationnel.
- Les métadonnées sont structurées : nom, type MIME, taille, token, dates et hash éventuel du mot de passe de partage.
- Les contraintes relationnelles, l'unicité de l'email et les migrations Flyway restent simples à maintenir.

Pourquoi pas MongoDB ?

MongoDB aurait pu convenir, mais n'apportait pas d'avantage décisif pour ce MVP très structuré. PostgreSQL rend la relation utilisateur-fichiers plus explicite.

Stockage - Système de fichiers local

Pourquoi je l'ai choisi : « Le stockage local suffit pour démontrer l'upload, le téléchargement et la suppression dans le MVP. Il évite d'ajouter la complexité d'AWS S3 au prototype. »

- Il suffit pour une démonstration locale et garde l'architecture du MVP simple.
- Il évite la gestion IAM, bucket et facturation cloud pendant le prototype.
- Limite connue : pour une application distribuée, un stockage objet comme AWS S3 serait plus adapté.

Outils techniques complémentaires

Outil	Rôle dans DataShare	Pourquoi il est utile
JWT	Authentification stateless	Protège les routes privées sans session serveur.
BCrypt	Hash des mots de passe	Évite le stockage des mots de passe en clair.
Maven	Build et dépendances	Standardise lancement, tests et plugins Java.
JaCoCo	Couverture backend	Mesure instructions, branches et lignes couvertes.
Playwright	Tests End-to-End	Vérifie le parcours utilisateur dans le navigateur.
k6	Test de charge backend	Mesure latence, taux d'erreur et débit.
Lighthouse	Performance frontend	Mesure FCP, LCP, TBT, CLS et score navigateur.
ESLint	Analyse statique frontend	Détecte erreurs et incohérences TypeScript.
SpotBugs	Analyse statique backend	Détecte des défauts potentiels dans le bytecode.
Flyway	Migrations PostgreSQL	Versionne le schéma ; Hibernate le valide.

API REST et sécurité

Endpoints principaux, contrôles d'accès et état de sécurité

3. Documentation des endpoints principaux

Méthode	Endpoint	Accès	Succès	Rôle
POST	/api/auth/register	Public	201	Créer un compte
POST	/api/auth/login	Public	200	Obtenir un JWT
POST	/api/files/upload	JWT	201	Téléverser un fichier
GET	/api/files	JWT	200	Historique paginé
DELETE	/api/files/{id}	JWT	204	Supprimer son fichier
GET	/api/download/{token}	Token	200	Métadonnées publiques
GET	/api/download/{token}/file	Token*	200	Télécharger le fichier

* Pour un fichier protégé seulement : en-tête X-Download-Password obligatoire. Absence ou erreur : 403. Pour un fichier non protégé, aucun mot de passe requis.

Exemples d'échanges

Connexion : POST /api/auth/login, JSON email + password ; succès : JWT.

Upload : POST /api/files/upload ; Bearer JWT ; multipart : file obligatoire, expirationDays (7 jours par défaut), password facultatif.

Téléchargement : GET /api/download/{token}/file ; contrôle du token, de l'expiration, des quotas et, si nécessaire, du mot de passe BCrypt.

4. Sécurité et gestion des accès

Authentification et partage

- BCrypt hache les mots de passe des comptes et des partages.
- JWT pour les routes privées ; vérification du propriétaire à la suppression.
- Lien public par token distinct du JWT, valable de 1 à 7 jours.
- Mot de passe de téléchargement facultatif.
- Fichier expiré : 410 ; introuvable : 404.

Contrôles et exploitation

- 1 Go maximum ; TXT, PDF, PNG, JPG, JPEG, MP3, MP4, ZIP ; contrôle des signatures.
- Rate limiting : connexion 10/min/IP ; upload 20/min/IP ; téléchargement 60/min/IP et 30/min/token (429).
- Secrets par variables d'environnement ; logs sans secrets.
- Stockage et compteurs mono-instance.

Audits des dépendances - 1er et 2 octobre 2026

Backend : Tomcat 11.0.24 : 11 CVE détectées ; après passage à 11.0.26 : 0 vulnérabilité détectée parmi les dépendances analysées (second scan OWASP ; base NVD en cache).

Frontend : npm audit --omit=dev : 7 alertes de production (3 modérées, 4 élevées), à traiter via une migration Angular préparée ; pas de fix --force.

Limites : ces audits ne démontrent pas l'absence totale de vulnérabilités. Voir SECURITY.md et API.md pour les détails techniques et les risques résiduels.

Qualité, maintenance, exécution et IA

Synthèse opérationnelle de la version validée le 6 octobre 2026

5. Qualité, tests et maintenance

58 / 58

tests backend

53 / 53

tests Angular

7 / 7

E2E Playwright

0

anomalie SpotBugs

Couverture JaCoCo backend - seuil vérifié : au moins 70 % des branches

Instructions : 91,64 % (1930/2106) Branches : 72,31 % (175/242) Lignes : 91,89 % (612/666)

Performance : mesures distinctes, environnements locaux

k6 - téléchargement backend

Campagne finale (6 octobre) : 4 liens, 31 requêtes par scénario ; 0 % d'erreurs.

Sans BCrypt : moyenne 5,50 ms ; p95 6,56 ms.
Avec BCrypt coût 12 : moyenne 215,05 ms ; p95 219,29 ms.
Surcoût moyen observé : environ 210 ms.

Lighthouse - frontend

Mobile : performance 91/100 ; LCP 2,9 s.
Desktop : performance 100/100 ; LCP 0,6 s.
Accessibilité, bonnes pratiques et SEO : 100/100 sur les deux profils.

Build Angular : 380,46 kB bruts ; transfert estimé 96,89 kB ;
aucun avertissement de budget.

Maintenance et sécurité

Purge des fichiers expirés ; sauvegardes PostgreSQL/fichiers à organiser ; Flyway V3 (owner_id, expires_at).
Hibernate ddl-auto=validate ; logs et audits réguliers. Voir MAINTENANCE.md et SECURITY.md.

6. Installation et exécution (Fedora)

1. Installer les prérequis (Java 21, Node.js/npm, PostgreSQL), puis initialiser PostgreSQL.

2. Configurer les secrets hors Git et charger source ~/.config/datashare/env.

3. Backend : cd backend && ./mvnw spring-boot:run.

4. Frontend : cd frontend && npm install && npm start ; interface : localhost:4200.

7. Utilisation de l'IA et supervision humaine

Démarche : cadrer le besoin, examiner la proposition, comprendre le code, tester, corriger et conserver une trace Git.

Exemple suivi : téléchargement par token ; un Content-Type invalide pouvait faire échouer une réponse. Ajout du repli application/octet-stream, puis test de non-régression.

L'IA a aussi assisté les modifications, les tests, les audits et la documentation ; l'intégration et la validation restent sous responsabilité humaine. Détails : AI_USAGE.md et AI_REVIEW.md.

Documents liés : README.md | API.md | TESTING.md | SECURITY.md | PERF.md | MAINTENANCE.md | PORTABILITY.md | PERSONAL_DATA.md

Note : k6, Angular et Playwright actualisés le 6 octobre ; Lighthouse reste la dernière campagne du 1er octobre.